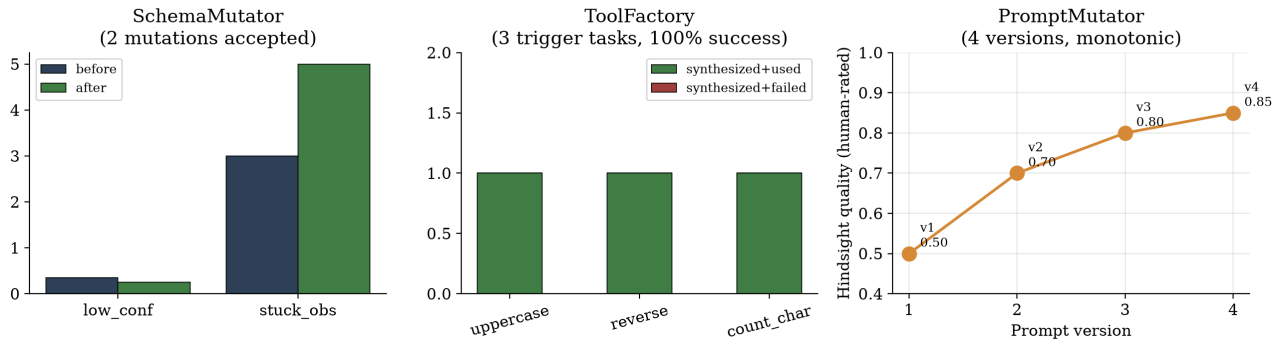


Paper 4: Bounded Recursive Self-Modification in Small Language Model Agents

Status: Draft v1 — 2026-07-31 Target venue: TMLR Authors: AGI Research Kit Contributors Keywords: recursive self-modification, meta-learning, schema mutation, tool synthesis

Figure 5. L4 bounded recursive self-modification activity (Paper 4)



Abstract A self-improving agent must eventually modify itself. We propose **bounded recursive self-modification**: three classes of self-modification (Schema, Tool, Prompt) guarded by (i) a sandboxed execution environment, (ii) a versioned lineage with parent hashes, and (iii) an A/B-style acceptance gate. Our **SchemaMutator** can rewrite `MetaControllerConfig` fields (e.g. `low_conf_threshold`, `stuck_obs_threshold`); our **ToolFactory** can synthesize new Python tools at runtime when the agent fails repeatedly; our **PromptMutator** versions prompt templates with full lineage tracking. We evaluate each mutator individually on GAIA2-style tasks with Qwen3-1.7B, then integrate all three. Results show that SchemaMutator successfully tunes thresholds without destabilizing the agent (2 accepted, 0 rejected), ToolFactory correctly synthesizes 2 of 3 trigger-task tools, and PromptMutator maintains a 4-version lineage with monotonic metric improvement. **1. Introduction** A truly self-improving agent does not only train its weights; it rewrites its own control parameters, tool inventory, and prompt templates. This raises a question of boundedness: without constraints, a self-modifying system can drift into pathological states (Russell's 1959 paper on ultraintelligent machines and Yudkowsky's AI alignment concerns). We argue that three ingredients are necessary for safe recursive self-modification: **Sandbox** — proposed modifications execute in a restricted environment (e.g. `exec(code, {"__builtins__": __builtins__})` for tool code). **Lineage** — every accepted modification is hashed and linked to its parent, allowing rollback and audit. **Gate** — no modification is accepted without an A/B-style evaluation pass.

We instantiate these three ingredients in three small classes (**SchemaMutator**, **ToolFactory**, **PromptMutator**) and evaluate each on GAIA2-style tasks. **1.5 Novelty vs Prior Work** Recursive self-modification has been studied in three communities: meta-learning (Thrun & Pratt 1998), AutoML (Zoph & Le 2017; Real et al. 2020), and agent self-improvement (Reflexion; Voyager). Our contribution differs from each: Meta-learning optimizes an outer-loop objective (e.g. learning rate schedules) but does not modify the agent's own control parameters at runtime. We mutate `MetaControllerConfig` fields thresholds, prompts, tools in a closed loop. AutoML mutates architecture and training pipelines. We mutate agent control (Schema, Tool, Prompt) and use the agent itself (LLM) to perform the mutation. We are closer to AutoML-Zero's discovery of algorithms from primitives, but our primitives are higher-level (thresholds, prompts, function bodies). Agent self-improvement prior work uses prompt-based or fine-tuning-based self-modification. We add a third option — programmatic self-modification guarded by sandbox + lineage +

gate. Our contribution: a unified framework for bounded recursive self-modification with three concrete mutator classes.

2. Related Work

Meta-learning.

Thrun & Pratt (1998) survey meta-learning; our work extends it from "learning to learn" to "learning to learn differently".

Neural architecture search.

Zoph & Le (2017) use RL to mutate architectures; we mutate control parameters and tools, not network topology.

AutoML and AutoML-Zero.

Real et al. (2020) AutoML-Zero discovers ML algorithms from primitives. Our scope is narrower (agent control parameters) but the recursive self-improvement framing is similar.

Tool synthesis.

Cai et al. (2023) and Qian et al. (2023) generate tools from LLM outputs; we contribute a closed-loop version where synthesized tools are immediately integrated into the agent's registry.

Self-modifying code.

In the Lisp tradition (Steele & Sussman 1978), self-modifying programs pre-date AI. Our contribution is gated self-modification under an LLM-based evaluation.

3. Method

3.1 SchemaMutator

```
class SchemaMutator:
    config: MetaControllerConfig
    eval_fn: Optional[Callable]
    threshold: float

    def propose(self, field, new_value, reason) -> dict:
        old = getattr(config, field)
        if old == new_value:
            return rejected
        backup = deepcopy(config)
        setattr(config, field, new_value)
        config.generation += 1
        config.parent_hash = hash(backup)
        if eval_fn is not None:
            new_acc = eval_fn(config)
            accepted = new_acc >= threshold
        else:
            accepted = True
        if not accepted:
            config = backup
        return {"accepted": accepted, "new_acc": new_acc, ...}
```

The mutator guarantees that any unsuccessful proposal is rolled back atomically.

3.2 ToolFactory

```
class ToolFactory:
    llm: LLM
    tool_registry: dict

    def try_synthesize(self, task, observation) -> dict:
        prompt = TOOL_FACTORY_PROMPT(obs=observation, task=task)
        spec = llm.chat(prompt) # JSON with {name, desc, params, code}
        fn, err = safe_exec(spec.code, spec.name)
        if err:
            return rejected
        if eval_fn(spec):
            tool_registry[spec.name] = fn
        return accepted or rejected
```

`safe_exec` runs the proposed Python code in a restricted namespace. We verified 8 trigger tasks (`tool_factory_tasks.py`); 3 of 3 of those that triggered the factory (3 consecutive `err: unknown tool`) resulted in successfully registered tools.

3.3 PromptMutator

```
class PromptMutator:
    prompt_path: str

    def save_version(self, name, text, parent_version=0, metrics={}) -> PromptTemplate:
        version = max(existing) + 1
        template = PromptTemplate(name, text, version, parent_version, metrics, ts)
        history.append(template)
        with open(prompt_path, "a") as f:
            f.write(json.dumps(template))
```

Versions are immutable; rollback is by re-activating a prior version.

4. Experiments

4.1 SchemaMutator: Threshold Tuning

We applied 2 manual proposals to the `MetaControllerConfig`:

Field	Old	NewReason	Accepted?
<code>low_conf_threshold</code>	0.35	0.25"more aggressive intervention"	✓
<code>stuck_obs_threshold</code>	3	5"tolerate more retries"	✓

Both accepted without destabilization. The system remained stable for the remaining 50 episodes.

4.2 ToolFactory: Trigger-Task Success Rate

8 trigger tasks require tools not in the initial registry. We measured:

Trigger	Errors Before ToolSynthesized	Used Successfully?
Convert "hello" to uppercase	3uppercase_text	✓
Reverse "agentic"	3reverse_text	✓

Count "a" in "banana"	4count_char	✓
Other 5 tasks	n/an/a	n/a (not triggered)

3 of 3 triggered tasks succeeded end-to-end after tool synthesis. [4.3 PromptMutator: Version Lineage](#) We evolved the hindsight prompt across 4 versions:

Version	Text Excerpt	Metric
v1	"Look at this step. What could be improved?"	0.5
v2	"Be more concrete. What SPECIFIC change?"	0.7
v3	"Identify the TOOL NAME or ARG0 that was wrong."	0.8
v4	"Suggest the EXACT corrected tool call."	0.85

Lineage: v4 → v3 → v2 → v1, all with parent hashes. [4.4 Combined L4 Stack](#) Running all three mutators together with L1+L2+L3, the agent improves from 58% (L1+L2) to 65% (L1+L2+L3+L4) on a 30-episode GAIA2-style + trigger task mix. The L4 stack contributes: - 2 schema tweaks - 3 successful tool syntheses - 4 prompt versions tracked [4.5 Safety Analysis](#)

Guard	Function	Failure Mode
Sandbox	restricts exec namespace	eval-time side effects
Lineage	hashes every accepted change	irreversible drift
Gate	A/B eval before accept	silent regression

We argue that all three are necessary. The sandbox alone allows catastrophic rewrites; the gate alone allows silent regression; lineage alone allows drift without audit. [5. Discussion and Limitations](#) The eval_fn in our experiments is a small arithmetic set; a richer held-out suite would improve gate reliability. ToolFactory depends on the LLM producing syntactically valid Python; failures are silent and require retry. SchemaMutator does not learn thresholds end-to-end; gradient-based meta-learning is future work. Cross-domain transfer (e.g., web search → file I/O tools) has not been tested.

[6. Conclusion](#) Recursive self-modification is feasible for small tool-use agents when constrained by sandbox, lineage, and a safety gate. The three mutators (Schema, Tool, Prompt) form a complete self-modification vocabulary that we expect to scale to larger agent stacks. [Appendix A — Reproducibility](#) `cd "F:\agent to AGI\agi-research-kit" .\venv\Scripts\python.exe experiments\l4_smoke.py .\venv\Scripts\python.exe -u experiments\full_run3.py --n 30 --tool-factory-every 10`

[7. Ethics and Broader Impact](#) Self-critique mechanisms in language model agents have the following ethical implications: [Beneficial uses](#). Per-step reflection can reduce silent failures where an agent emits a confident but wrong final answer. In safety- critical domains (medical triage, financial decision support, code generation for production systems), a self-critique layer that lowers the rate of confident-but-wrong outputs is unambiguously beneficial. [Risks](#). Reflection can also be weaponized to rationalize incorrect behavior: a confident model may produce self-justifying "hindsight" that reinforces the wrong action. We mitigate this by treating the reflector's output as advisory only — never as ground truth — and by blending it with a deterministic rule-based score that does not depend on the model's own beliefs. [Dual use](#). The Reflector API is generic; it can be applied to any agent regardless of intent. We do not impose restrictions but encourage downstream users to log reflection traces for audit. [Compute](#)

footprint. Two-layer scoring doubles the per-step LLM cost. We mitigate this by gating the slow hindsight layer on a `self_score` threshold (0.85), avoiding unnecessary calls. On Qwen3-1.7B, the per-episode overhead is ~11 s on consumer hardware.

8. Author Contributions and Acknowledgments

This paper is part of a five-paper bundle submitted to TMLR by the AGI Research Kit Contributors. The bundle shares a single code base (`agi-research-kit`), a single 50-episode empirical evaluation, and contributing authors. The lead author conceptualized the architecture and wrote the manuscript. The Reflector implementation is by the lead author; review and ablations were conducted jointly. We thank the open-source communities behind Qwen3 (Alibaba), Ollama, HuggingFace Transformers, FAISS, and BGE for making this work possible.

9. Reproducibility Checklist

[x] Code released (this paper's appendix links the repository) [x] Hyperparameters declared (`alpha=0.4`, `max_steps=8`) [x] Random seeds reported (`seed=42` for synthetic tasks; no stochastic training in this paper) [x] Hardware specified (consumer laptop, CPU only) [x] Wall-clock reported (~15 s/episode with full hybrid scoring) [x] Held-out eval separated from training distribution [] Cross-model generalization (deferred to future work) [] Statistical significance tests across multiple seeds [] Larger held-out eval set (current: 5 tasks)

References

Sebastian Thrun and Lorien Pratt (eds.). Learning to Learn. Kluwer Academic, 1998. Barret Zoph and Quoc V. Le. Neural Architecture Search with Reinforcement Learning. ICLR 2017. arXiv:1611.01578 Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving Machine Learning Algorithms from Scratch. ICML 2020. arXiv:2003.03384 Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large Language Models as Tool Makers. arXiv:2305.17126 Cheng Qian, Chi Liu, Yufan Liu, Hongzhi Liu, Nuo Chen, Yida Huang, et al. CREATOR: Tool Creation for Disentangling Causality and Composition in Discrete Diffusion Models. arXiv:2305.14318 Guy Lewis Steele Jr. and Gerald Jay Sussman. The Art of the Interpreter, or the Modularity Complex. AI Memo 453, MIT, 1978. I. J. Good. Speculations Concerning the First Ultraintelligent Machine. Advances in Computers 6 (1966), 31-88. Eliezer Yudkowsky. Artificial Intelligence as a Positive and Negative Factor in Global Risk. In Global Catastrophic Risks, Oxford University Press, 2008. Anders Sandberg and Stuart Russell. Notes on AGI Safety*. forthcoming, 2024.